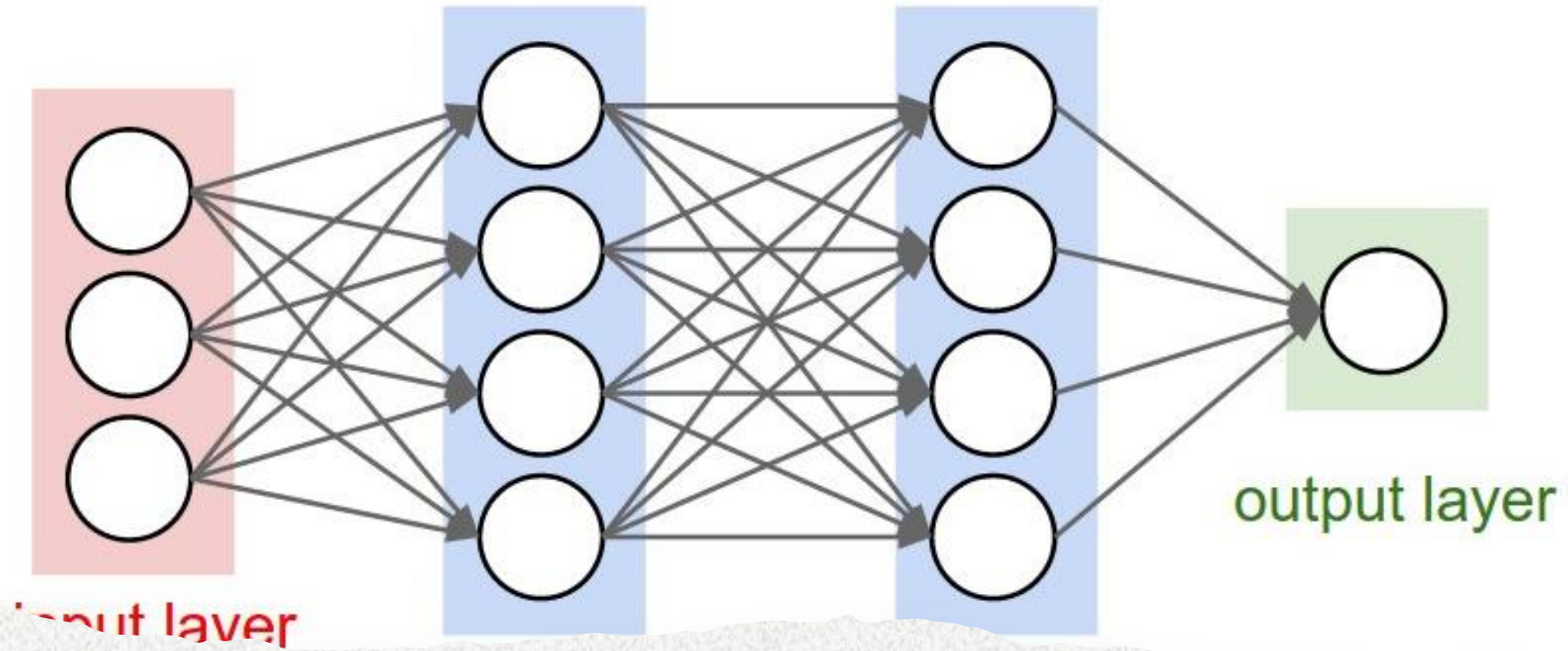
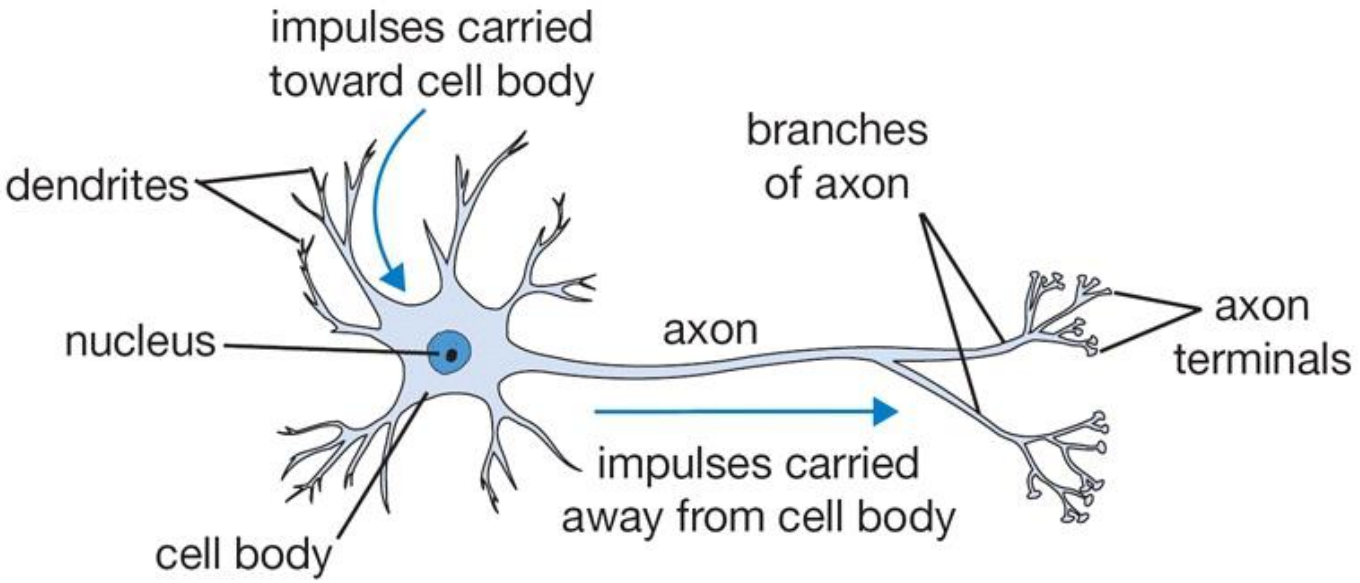


Neuronske mreže

Feedforward neural networks

Biološka motivacija

- Osnovna procesna jedinica ljudskog mozga je neuron
  - 10^{11} neurona
  - $10^{14} - 10^{15}$ sinapsi (svaki neuron je povezan sa oko $10^3 - 10^4$ drugih neurona)
  - Do neurona preko dendrita dolaze impulsi
    - Impulsi povećavaju ili smanjuju verovatnoću da će neuron proizvesti impuls na aksonu



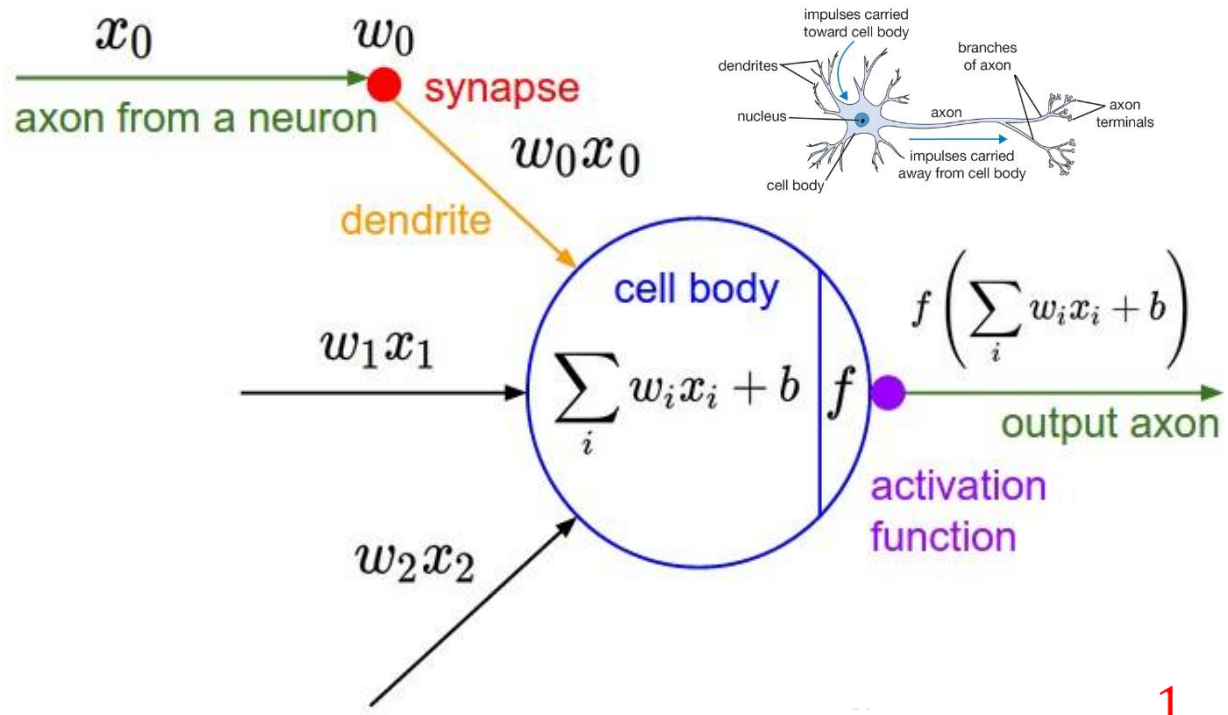
Matematički model veštačkog neurona

Nacrtajte matematički model veštačkog neurona.

Označite sve prikazane elemente.

Prikažite kako se izlaz neurona računa na osnovu ulaza.

Matematički model veštačkog neurona



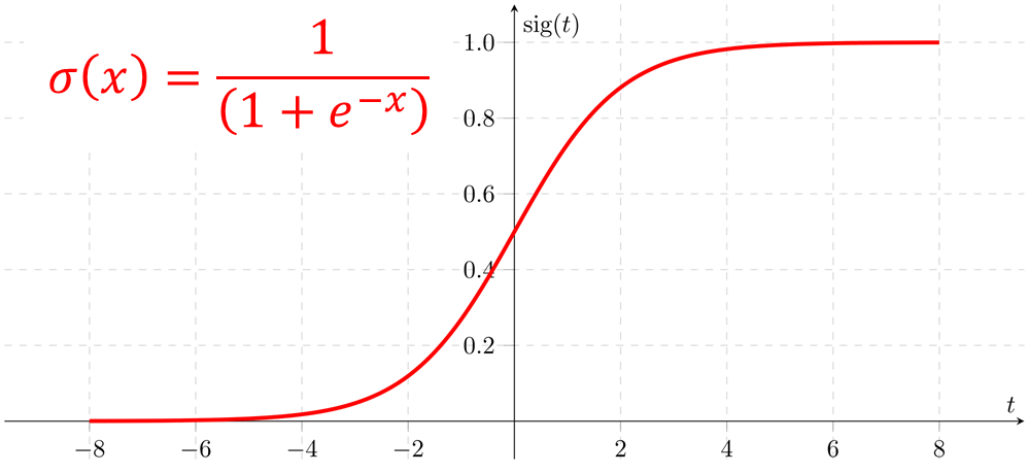
f – aktivaciona funkcija

- Određuje nivo pobuđenosti neurona za date ulaze
- Postoji više tipova aktivacionih funkcija

Primer aktivacione funkcije: sigmoidna funkcija

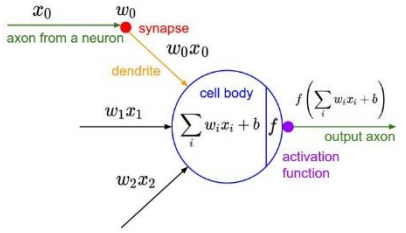
transformiše $\sum_i w_i x_i + b$ iz opsega $(-\infty, +\infty)$ u opseg $[0,1]$

$$\sigma(x) = \frac{1}{(1 + e^{-x})}$$



Analogija prirodnog i veštačkog neurona

- Budite veoma pažljivi u pravljenju analogije između veštačkog i biološkog neurona!

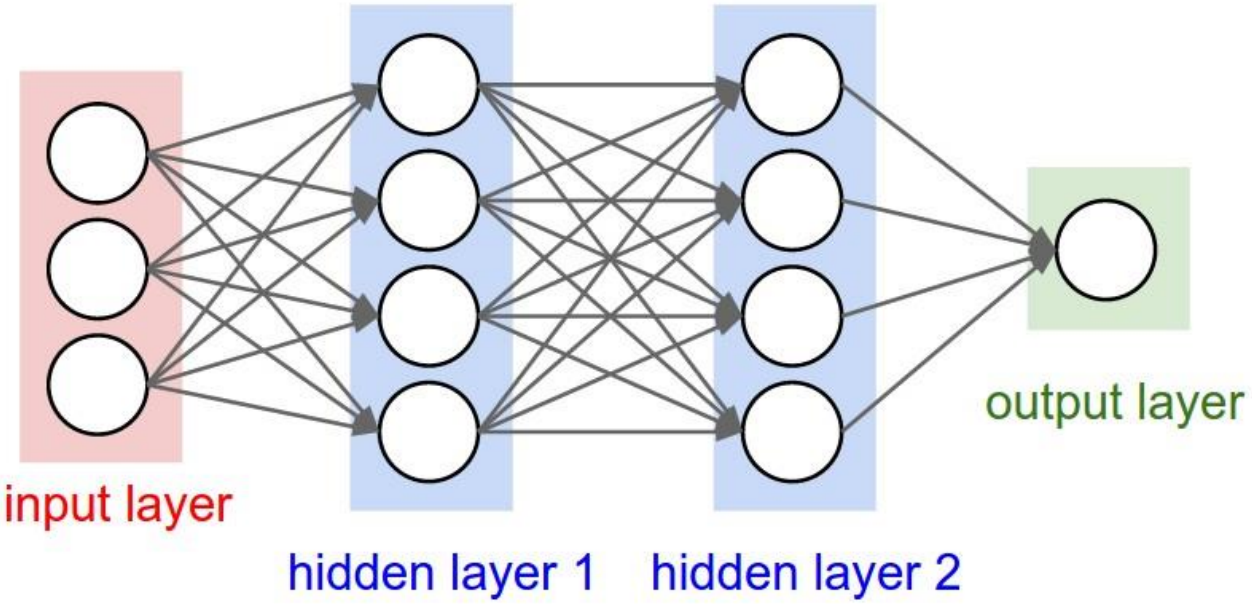


- Biološki neuroni
  - Postoji više različitih tipova
  - Dendriti mogu da primenjuju kompleksne nelinearne funkcije
  - Aktivacionu funkciju obično „interpretiramo“ kao frekvenciju okidanja
    - Intuicija je da neuroni prenose signale povezanim neuronima u diskretnim impulsima – ako je frekvencija velika, signal je snažan
    - Od aktivacionih funkcija, najpribližnija aproksimacija je ReLU
    - Ali, posmatranje isključivo frekvencije impulsa verovatno nije dovoljno. Kod biološkog neurona impulsi nisu u ravnomernim vremenskim razmacima i ovo je verovatno od značaja

Organizacija po slojevima

Objasnite kako su neuroni organizovani u potpuno povezanoj veštačkoj neuronskoj mreži *feedforward* arhitekture.

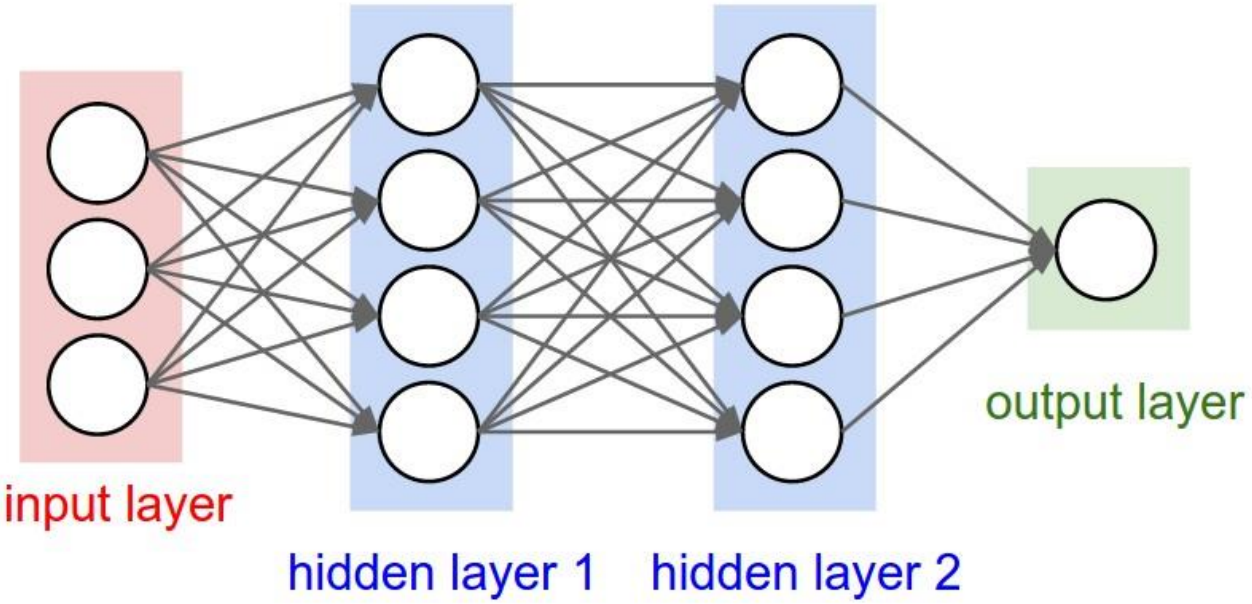
Organizacija po slojevima



- U neuronskoj mreži neuroni su grupisani u slojeve
  - Prvi sloj je ulazni i razlikuje se od ostalih: prosleđuje vrednosti samog uzorka
  - Poslednji sloj je izlazni i njegov izlaz predstavlja rezultat
  - Slojevi između ulaznog i izlaznog nazivaju se skriveni slojevi

- Na slici je dat primer neuronske mreže sa **tri** sloja
  - dva skrivena i jedan izlazni (ne brojimo ulazni sloj)

Feedforward arhitektura



- Prikazana je struktura sa propagacijom unapred (*feedforward*)
  - Izlazi neurona su povezani *isključivo* na neurone sledećeg sloja

- Slojevi su međusobno potpuno povezani (*fully connected layer*)
  - Neuroni jednog sloja su povezani sa svim neuronima sledećeg sloja

Model

- Šta su parametri, a šta hiper-parametri modela neuronske mreže?

- Parametri modela (koji se uče iz podataka):
  - Težine veza ω
  - Bias b

- Hiper-parametar: arhitektura
  - Broj skrivenih slojeva
  - Broj neurona u svakom od skrivenih slojeva
  - Aktivaciona funkcija

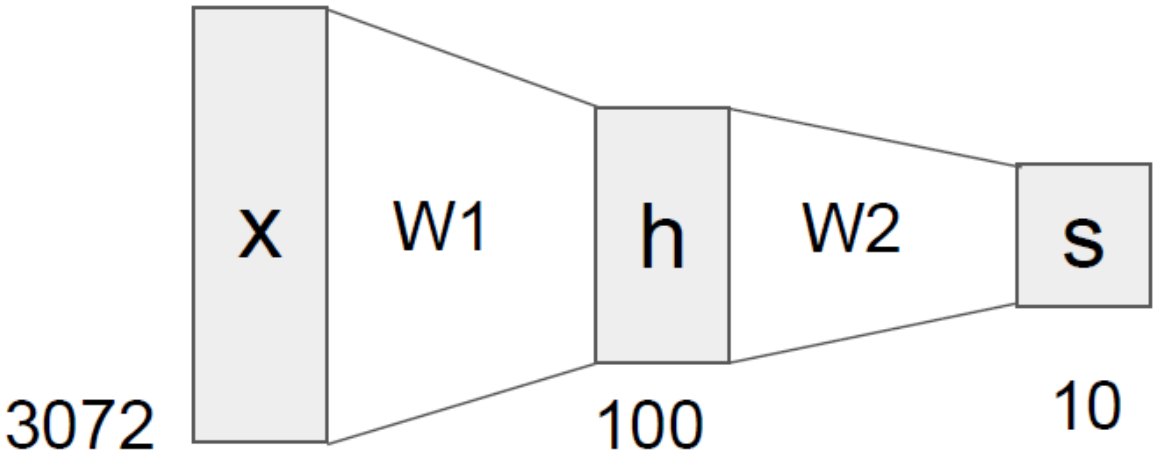
Predikcija modela (*feedforward*)

- Kako za dati ulaz x_i odrediti izlaz (predikciju $h(x_i)$)?

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$
or 3-layer Neural Network

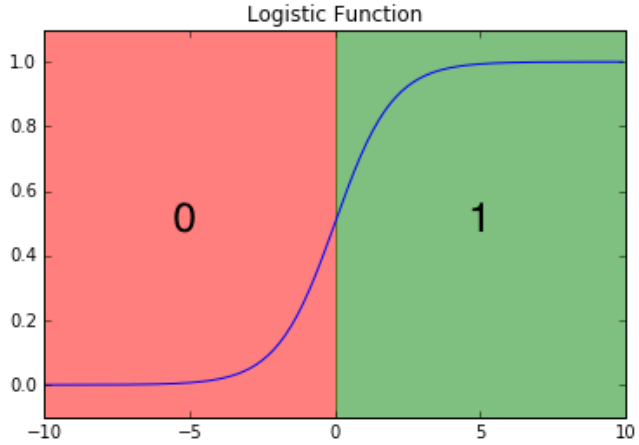
$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

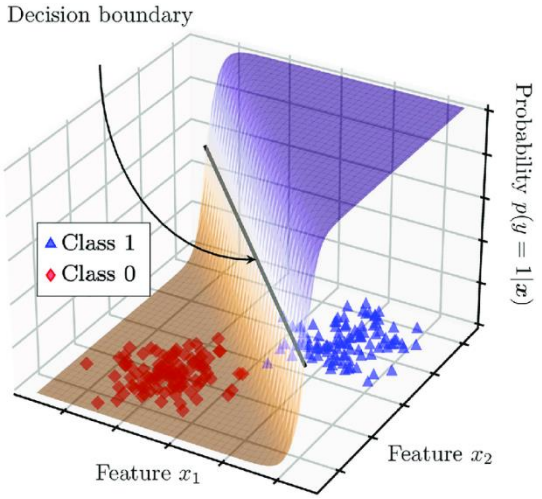


Kombinovanje neurona

Zašto u neuronskoj mreži kombinujemo više neurona?

Odnosno, šta bismo imali da smo ograničeni isključivo na jedan neuron?





Šta dobijamo sa jednim neuronom bez aktivacione funkcije?

Benefiti slaganja neurona u slojeve:

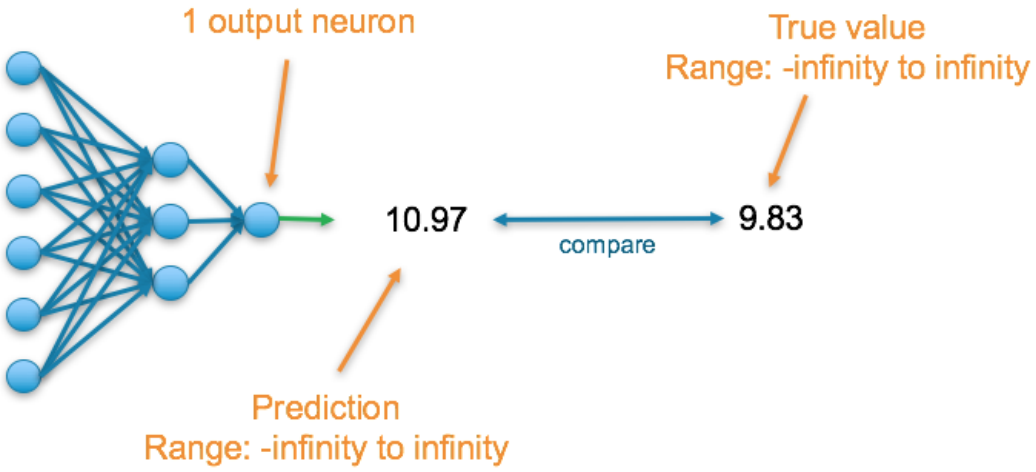
1. Predstavljanje kompleksnih, nelinearnih funkcija
2. Hijerarhija obeležja
3. Efikasnost i paralelizam

Izlaz u slučaju regresije

- U poslednjem sloju je samo jedan neuron
  - Ne koristimo aktivacionu funkciju na ovom neuronu

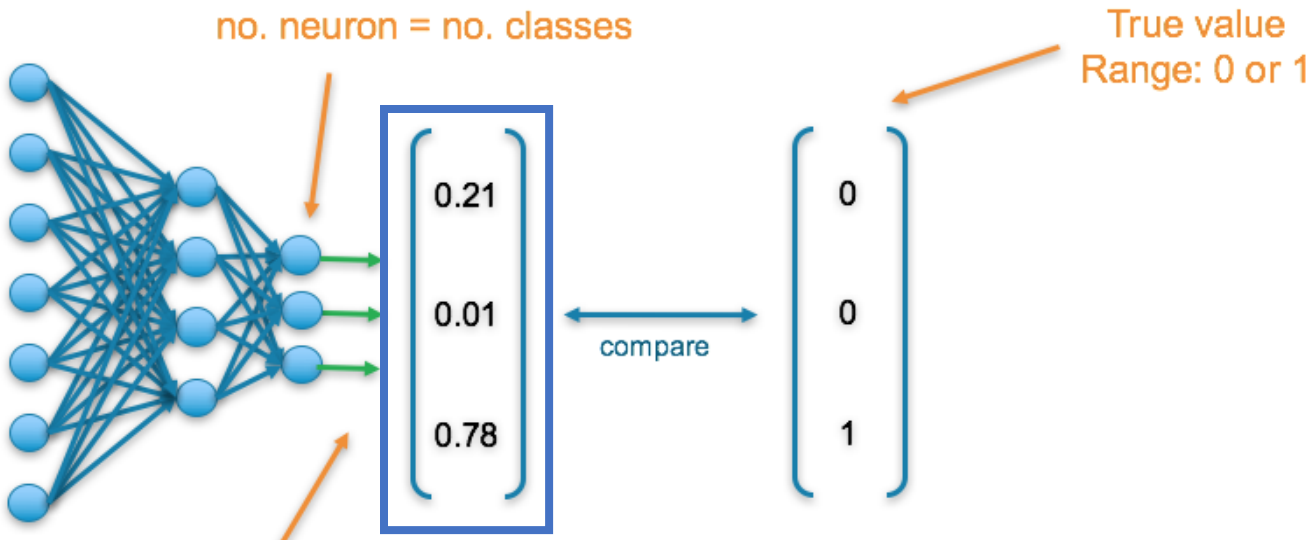
- Izlaz (naša hipoteza $h(x_i)$ je izlazni signal iz ovog neurona

- Greška predikcije je *Mean Squared Error* (MSE):
  - $(h(x_i) - y_i)^2$, gde y_i predstavlja stvarnu labelu primera x_i



Klasifikacija

- Broj neurona u poslednjem sloju je jednak broju kategorija (klasa) K
- Obično ne koristimo aktivacione funkcije na ovim neuronima



Verovatnoća svake od *K* klasa → transformišemo izlazne signale da budu ≥ 0 i da se sumiraju na 1 → *Softmax*

Klasifikacija - *Softmax*

- Izlazni signali: $s_1, s_2,\ldots,s_K$
- Izlaz: *Softmax* funkcija

$$P(y_i = k|X = x_i) = \frac{\exp s_k}{\sum_{i=1}^{K} \exp s_i}$$

- Greška: negativna vrednost logaritma verodostojnosti paremetara

$$L_i = -\log P(Y = y_i|X = x_i)$$

Klasifikacija

$$L_i = -\log P(Y = y_i | X = x_i)$$

Softmax Classifier (Multinomial Logistic Regression)

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$



unnormalized probabilities

cat	**3.2**	exp	**24.5**	normalize	**0.13**
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

unnormalized log probabilities

probabilities

L_i = -log(0.13)
= 0.89

Klasifikacija - gubitak

Koje su tipične metrike koje koristimo za evaluaciju performansi klasifikacionog modela?

Za treniranje koristimo funkciju gubitka, a rezultate modela ocenjujemo koristeći metrike poput tačnosti ili F-mere

1. Funkcija gubitka:
    1. Kontinualnost i diferencijabilnost. Metrike za evaluaciju modela mogu imati nediferencijabilan ili neinformativan gradijent
    2. Efikasnost računanja
    3. Stabilnost obuke
    4. Nebalansiranost, generalizacija
2. Tačnosti/F-mera:
    1. Bolja interpretabilnost u smislu performansi modela

Obučavanje modela

- Dakle, za dat ulaz x_i i za date težine w možemo da izračunamo šta je predikcija modela $h_w(x_i)$
- Šta učimo (šta su parametri modela)?
  - Težine w u modelu (uključujući *bias*-e)
  - Obučavanje modela: pronalazak težina w tako da se predikcije $h_w(x_i)$ što bolje poklapaju sa stvarnim vrednostima izlaza y_i

- Kako ih učimo?
  - Izračunaćemo grešku predikcije
    - Razlika stvarnih vrednosti i onoga što naš model predviđa
    - Zavisi od parametara modela w
  - Minimizovaćemo grešku po w

Funkcija greške

- Za instancu x_i:
  - $h_w(x_i)$ - predikcija, y_i - stvarna labela
  - Greška predikcije na datoj instanci: $e(h_w(x_i), y_i)$
  - Npr. kod regresije, greška predikcije može biti MSE

$$e(h_w(x_i), y_i) = (h_w(x_i) - y_i)^2$$

- Za trening skup (N instanci):

$$E(w) = \frac{1}{N}\sum_{i=1}^{N} e(h_w(x_i), y_i)$$

(E je funkcija od w – obučavajući primeri $\{(x_i, y_i)\}$ su fiksni, ono što je promenljivo u modelu su težine w)

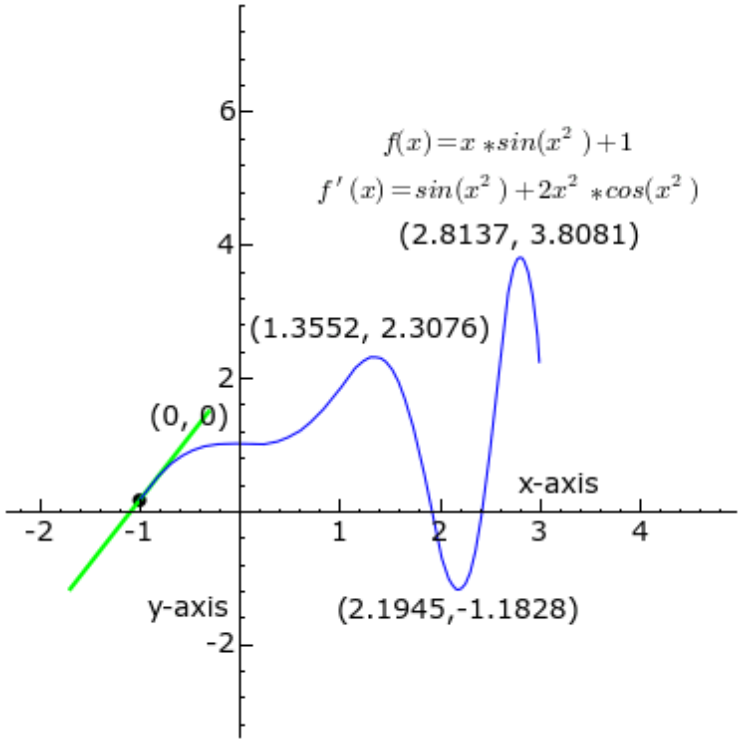
- Želimo da nađemo w za koje je $E(w)$ minimalno

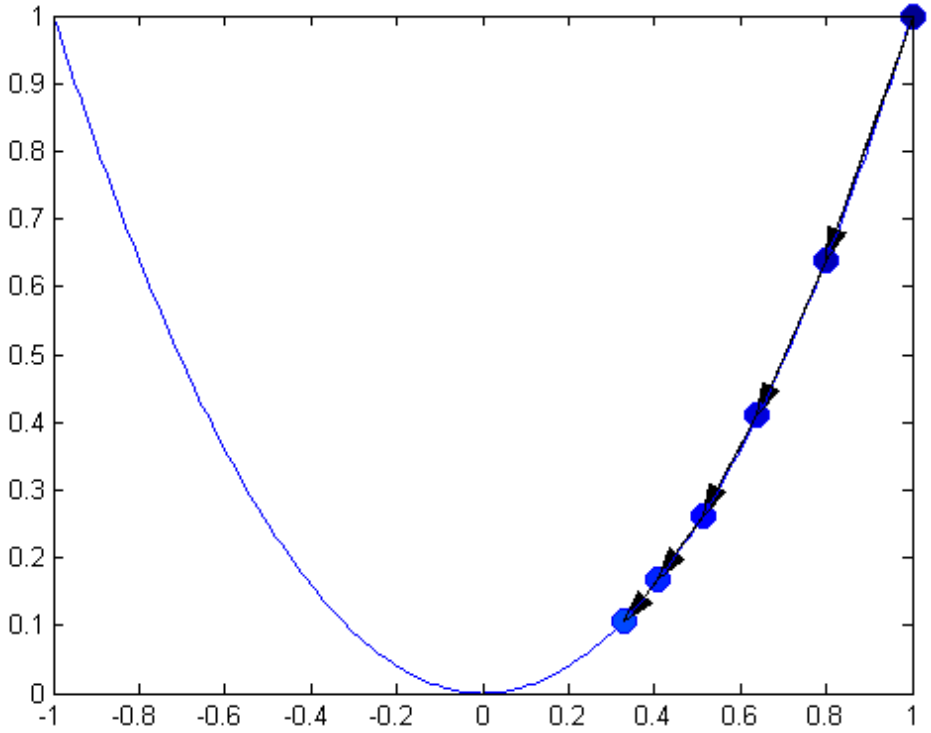
Funkcija greške

- Kako minimizovati funkciju greške?
  - Parametri bi se mogli varirati na razne načine kako bi se videlo u kom pravcu se može izvršiti pomak kako bi se dobila manja vrednost greške
  - Takav postupak je računski skup
  - Gradijent daje pravac pomeranja u kojem se greška *lokalno* najbrže povećava
  - Algoritam za optimizaciju: gradijentni spust

Gradijentni spust (*Gradient Descent*, GD)

- Izvod funkcije predstavlja nagib tangente na krivu funkcije
- Ideja: iterativno ćemo se pomerati ka minimumu
  - levo od minimuma: gradijent je negativan – pomeramo se u desno
  - desno od minimuma: gradijent je pozitivan – pomeramo se u levo





Gradijentni spust (*Gradient Descent*, GD)

- GD algoritam:
  - Inicijalizovati $w(0)$
  - Za $t = 1,2, \ldots$ (do konvergencije)
    - $w(t+1) = w(t) - \alpha \nabla_w E(w)$
  - Rezultat: optimalno w

GD_visualization.m

- Problem: $E(w) = \frac{1}{N} \sum_{i=1}^{N} e(h_w(x_i), y_i)$
  - Moramo proći kroz sve instance trening skupa: *Batch GD*
  - Loše za veliko N – mnogo računanja pre nego što se pomerimo
- Rešenje:
  - Aproksimiraćemo sumu korišćenjem podskupa trening skupa: *Minibatch GD*
  - Tipično: 32, 64 ili 128 primera

https://blog.paperspace.com/intro-to-optimization-in-deep-learning-gradient-descent/

Obučavanje modela – *Mini-batch SGD*

- Uzmemo podskup primera trening skupa $\{x_i\}$
- Propustimo primere $\{x_i\}$ kroz neuronsku mrežu da dobijemo predikcije $\{h_w(x_i)\}$
- Koristimo funkciju greške da Izračunamo gubitak (*loss*)

$$E(w) = \frac{1}{|\{x_i\}|} \sum_i e(h_w(x_i), y_i)$$

- Ažuriramo w u pravcu negativnog gradijenta greške $E(w)$

Računanje gradijenta kod NN

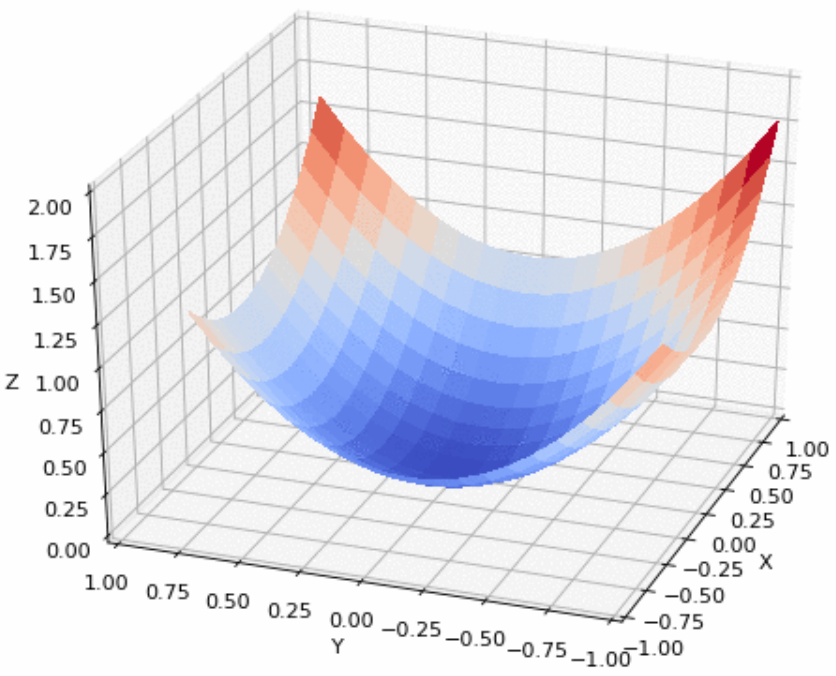
- Dakle, ono što nam treba da primenimo GD jeste da odredimo gradijent funkcije greške $\nabla_w E(w)$
  - Ovo nije teško: neuronske mreže definišu kompleksnu funkciju greške, ali to nam je poznata funkcija
  - Npr. u slučaju MSE greške $E(W) = \sum_i (h_w(x_i) - y_i)^2$, a $h_w(x_i)$ računamo rekurzivno: to je izlazni funkcija poslednjeg sloja koji je funkcija sloja pre njega, a koji je funkcija sloja pre njega...
  - Dakle, analitička forma funkcije $E(W)$ je poznata i ne bi trebalo da predstavlja problem da odredimo gradijent
  - Problem je – kako da ovo uradimo *efikasno*?

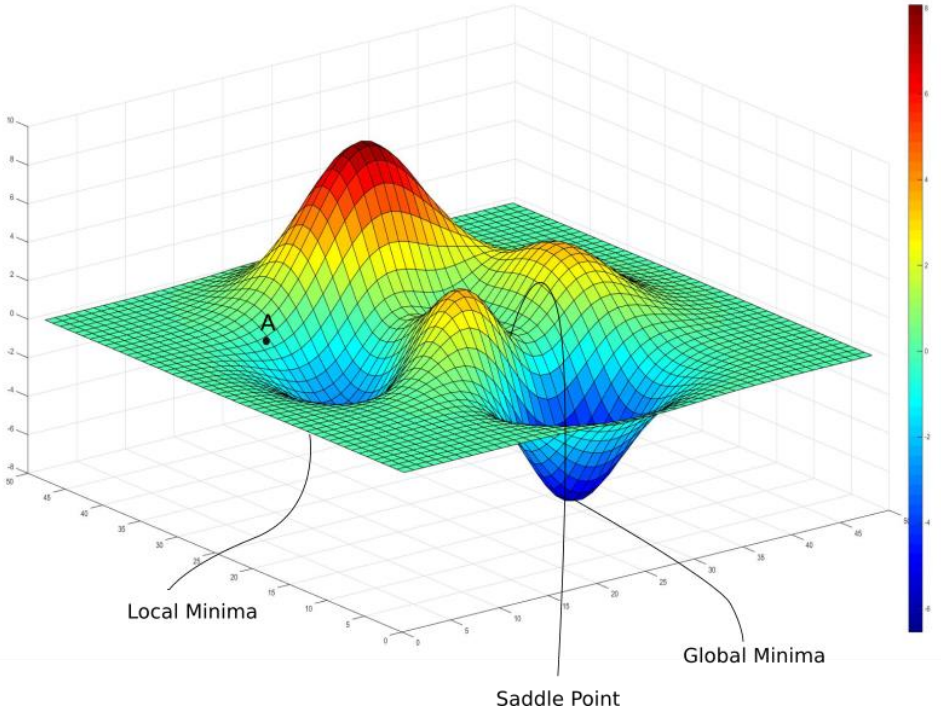
- Algoritam propagacije u nazad (*backpropagation*)
  - Rekurzivna primena pravila pronalaska izvoda složene funkcije (*chain rule*)

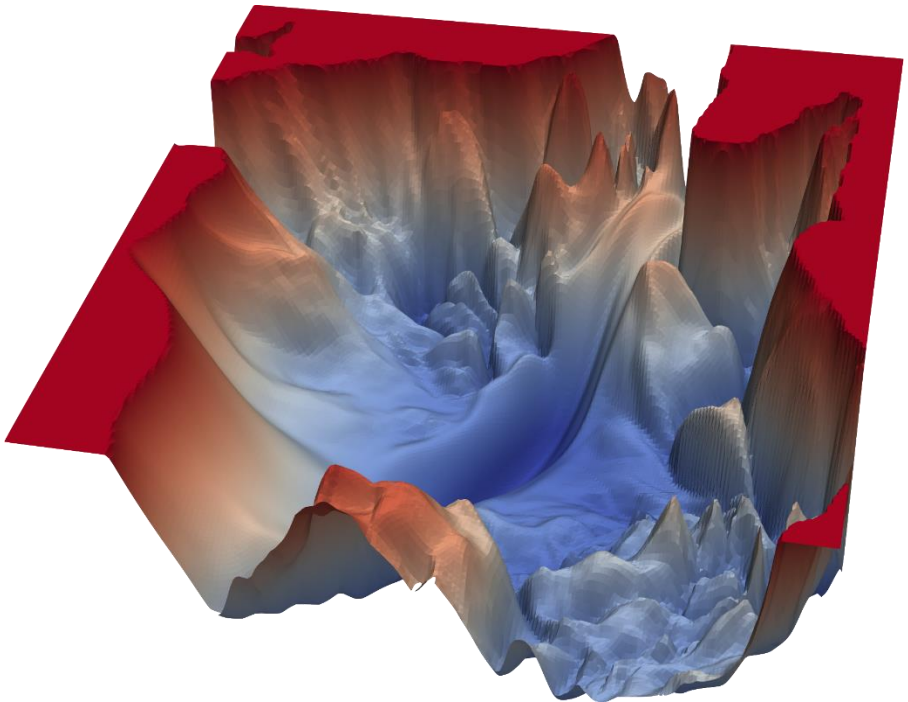
$$\big(f(g(x)\big)' = f'\big(g(x)\big)g'(x)$$

Backpropagation

- CS231N
  - Lecture 4: *Introduction to Neural Networks*
    - https://www.youtube.com/watch?v=d14TUNcbn1k&index=4&list=PL3FW7Lu3i5JvHM8ljYj-zLfQRF3EO8sYv
- https://www.youtube.com/watch?v=Ilg3gGewQ5U





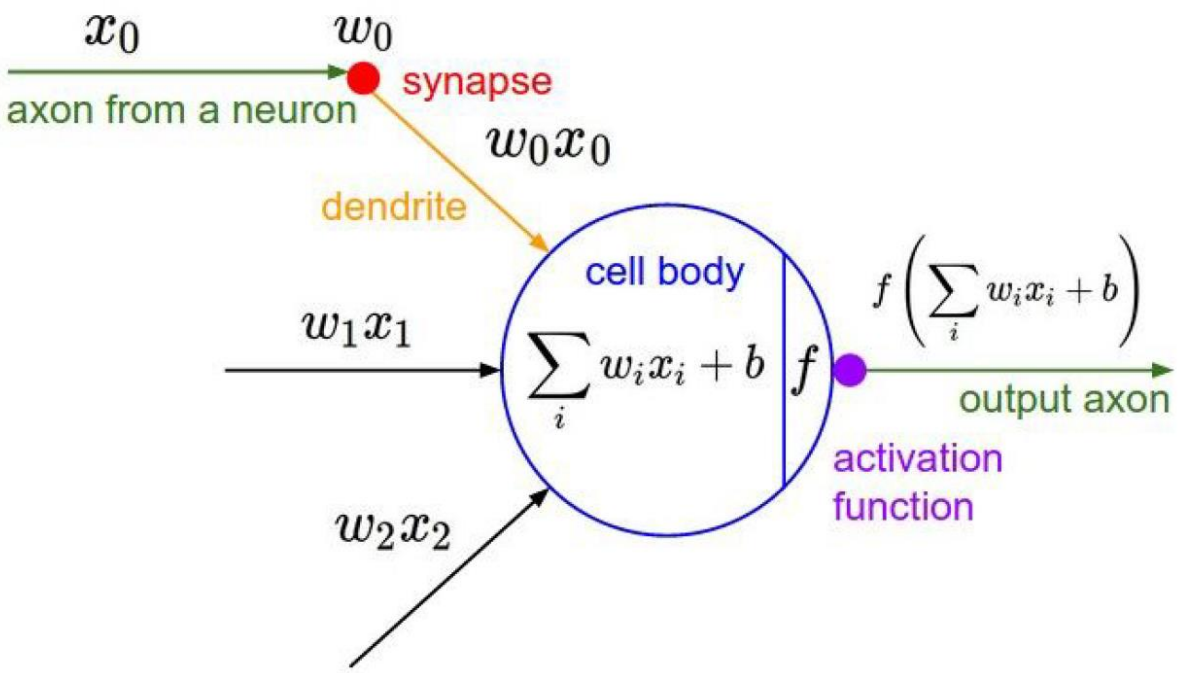


A Complicated Loss Landscape *Image Credits:* *https://www.cs.umd.edu/~tomg/projects/landscapes/*

Li, H., Xu, Z., Taylor, G., Studer, C. and Goldstein, T., 2018. Visualizing the loss landscape of neural nets. *Advances in neural information processing systems*, *31*.

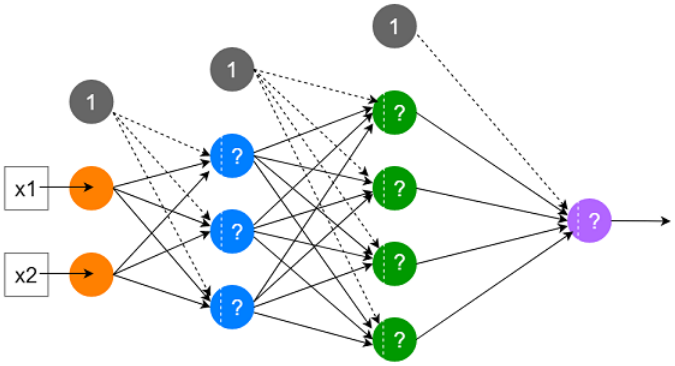
Aktivacione funkcije

Activation Functions



Zašto aktivaciona funkcija?

- Ako ne koristimo aktivacionu funkciju, izlazni signal će biti linearna funkcija
- Kombinacija linearnih funkcija je takođe linearna funkcija
- Bez aktivacione funkcije, cela neuronska mreža bi se svela na (veoma komplikovanu) linearnu regresiju



Hidden layer 1: Here, the input is X.

```
X*W1 + b1
```

Hidden layer 2: Here, the input is X*W1 + b1.

```
(X*W1 + b1)*W2 + b2
```

Output layer: Here, the input is (X*W1 + b1)*W2 + b2.

```
((X*W1 + b1)*W2 + b2)*W3 + b3
```

The final output is ((X*W1 + b1)*W2 + b2)*W3 + b3. We can simplify this expression further.

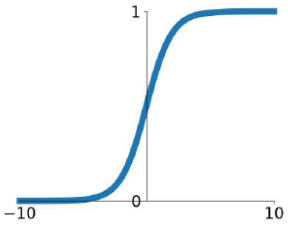
```
(X*W1*W2+ b1*W2 + b2)*W3 + b3
(X*W1*W2*W3 + b1*W2*W3 + b2*W3) + b3
(X*W1*W2*W3) + (b1*W2*W3 + b2*W3 + b3)
```

Aktivacione funkcije

Activation functions

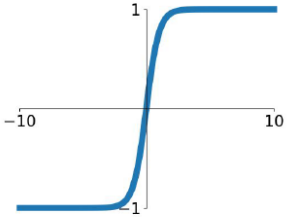
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



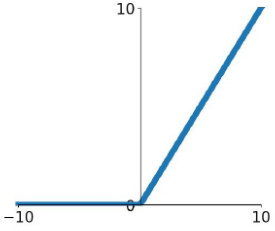
tanh

$$\tanh(x)$$



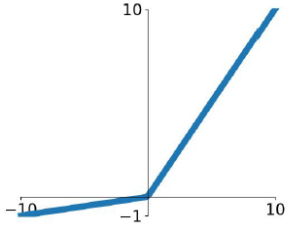
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

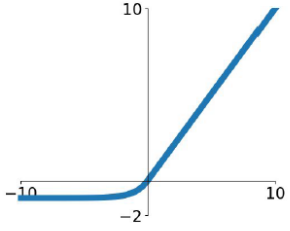


Maxout

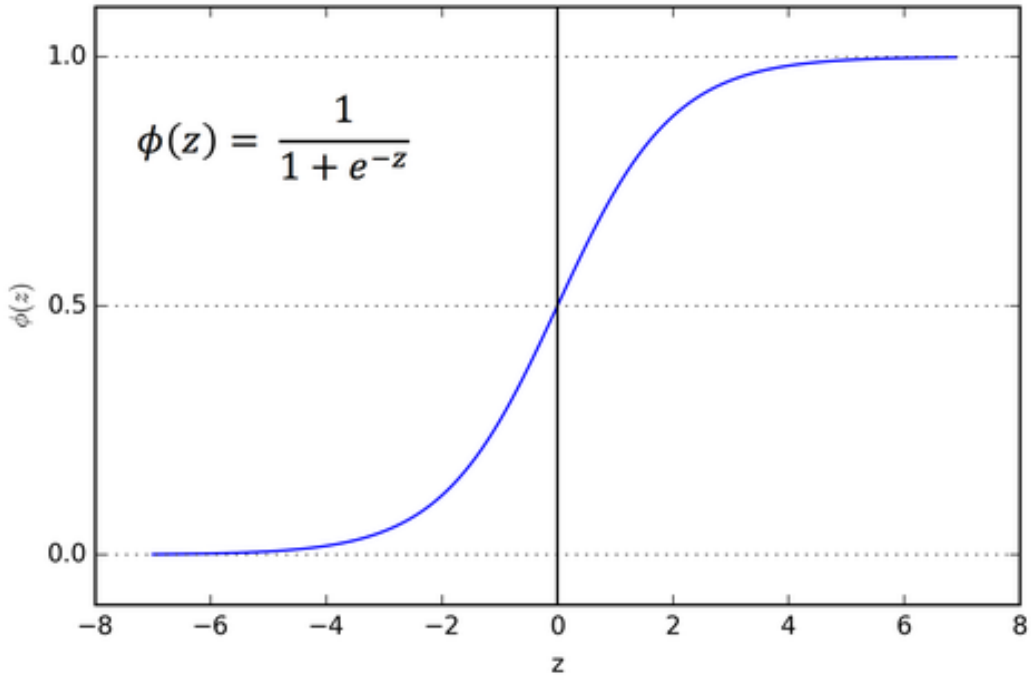
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Sigmoid



- Mapira izlazni signal na interval $[0,1]$
- Istorijski popularna zbog lepe interpretacije

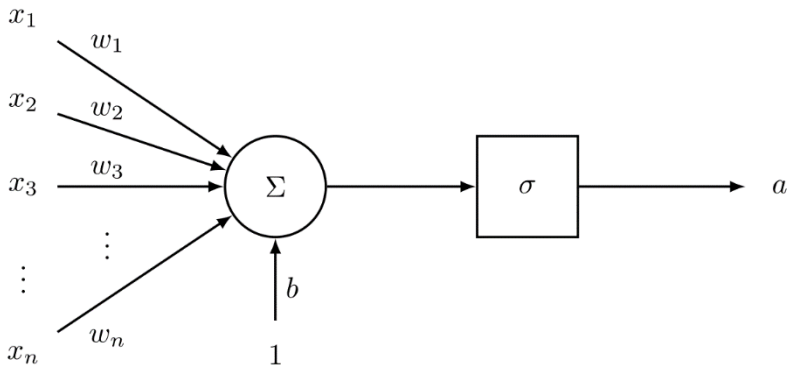
Sigmoid – problemi

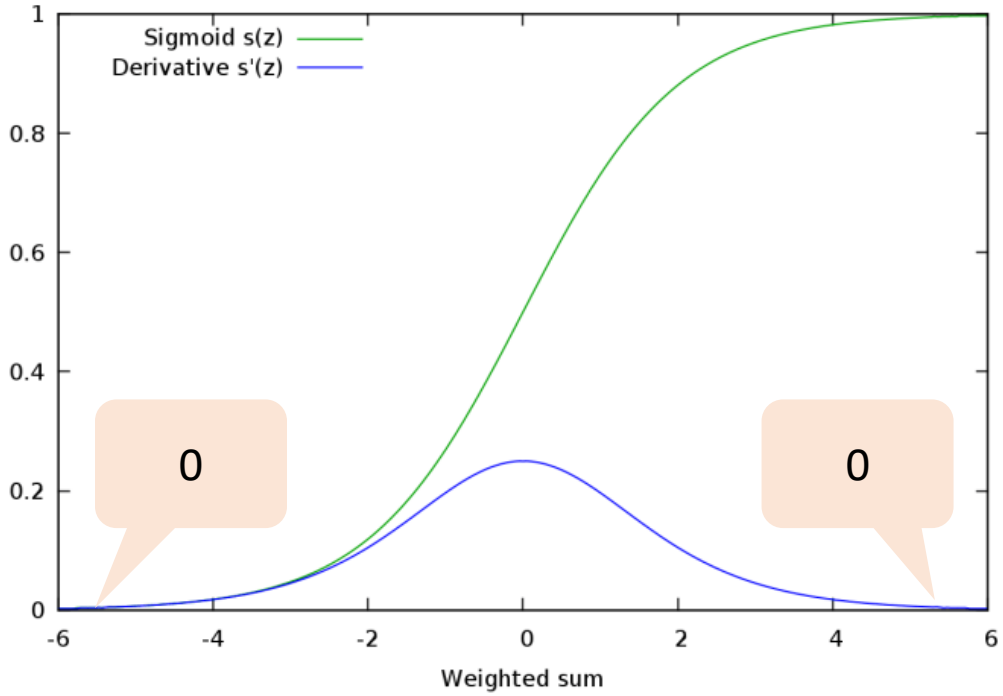
1. Ubija“ gradijente saturisanih neurona (*Vanishing Gradients*)

2. Nije centrirana oko nule

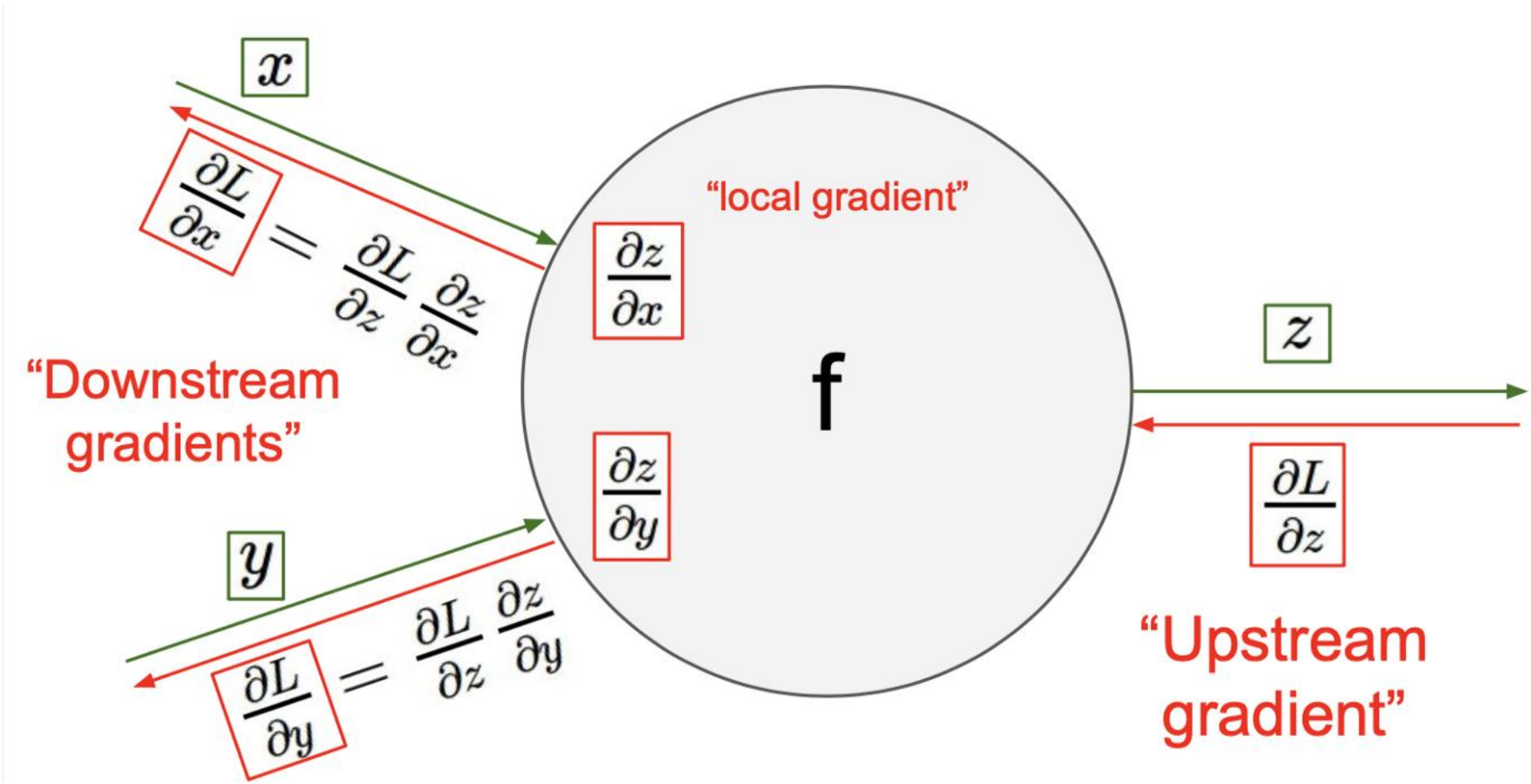
3. Eksponencijalna funkcija je skupa za računanje

Vanishing Gradients





Vanishing Gradients



Backpropagation: a simple example

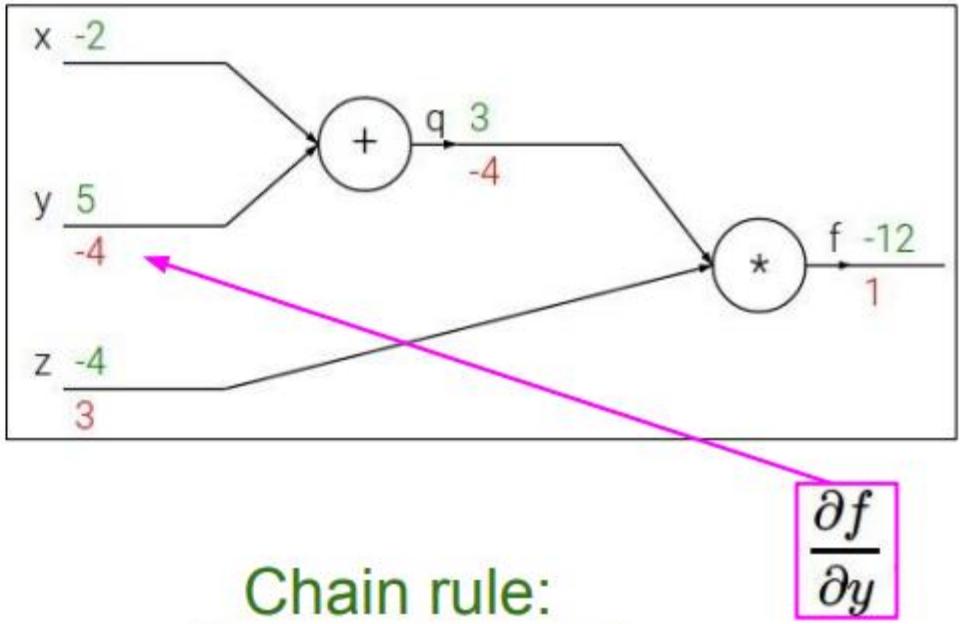
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

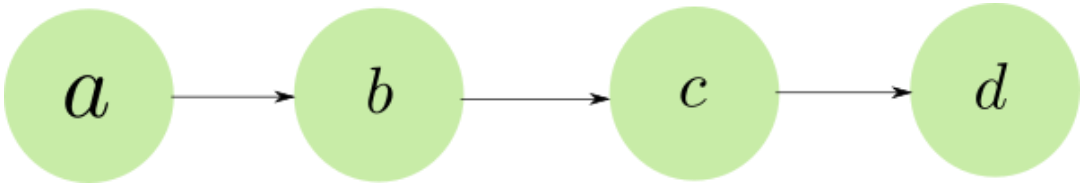
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Vanishing Gradients

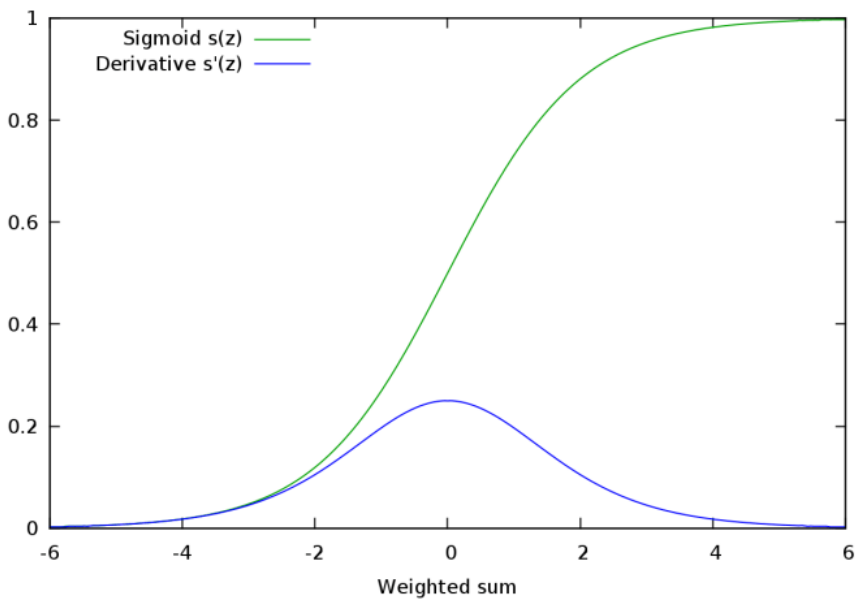


$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial d} * \frac{\partial d}{\partial c} * \frac{\partial c}{\partial b} * \frac{\partial b}{\partial a}$$

- Zamislite da imamo 50 neurona pre *a*
- To znači da množimo sa 50 gradijenata sigmoida, gde je svaki broj između 0 i 1
- Možete isprobati eksperiment – nasumično odaberite brojeve između 0 i 1 i pomnožite ih – rezultat će biti reda veličine 10^{-18}

Vanishing Gradients

- Gradijent u dubljim slojevima mreže postaju 0 („nestaju“)
- Dublji slojevi mreže će se ažurirati veoma sporo ili, u gorem slučaju, neće se ni obučavati
- Slučaj postaje gori za saturisane neurone (lokalni gradijent blizak 0)
- Donekle možemo rešiti pažljivom inicijalizacijom ω – ako su vrednosti prevelike, većina neurona će postati saturisana i mreža će jedva da

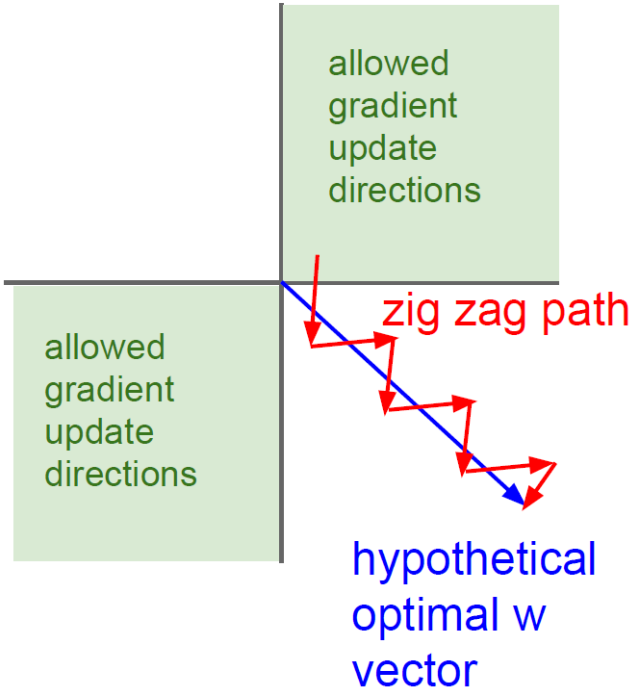


Prepoznavanje problema

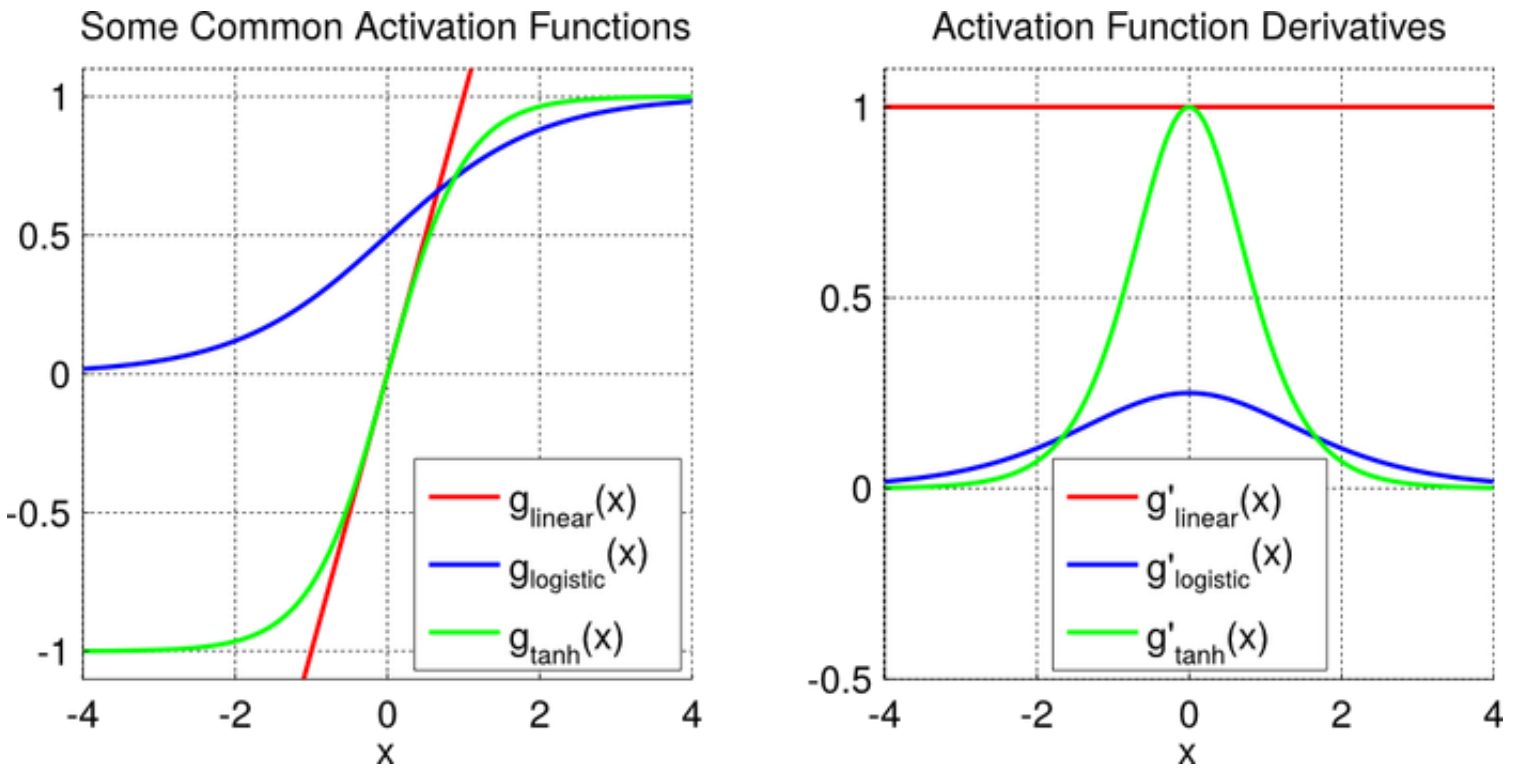
- Velike promene su primetne u težinama slojeva bliskih izlazu, dok se parametri prvih slojeva malo menjaju ili ostaju nepromenjeni.
- Nekada težine prvih slojeva mogu postati 0 kako trening odmiče.
- Model se trenira sporo i često se trening zaustavlja nakon nekoliko iteracija.
- Performanse modela su loše

Sigmoid – problemi

- Izlaz iz sigmoidne funkcije nije centriran oko nule
  - Ulaz u naredni neuron je uvek pozitivan
- $f(\sum_i w_i x_i + b)$ - ako je svako x_i pozitivno, lokalni gradijenti $\partial f / \partial w_i$ su svi pozitivni
- Gradijenti w su svi pozitivni (ili svi negativni)
  - zavisno od gradijenta koji nam se vraća sa izlaza
  - Zato će se svi gradijenti w kretati u istom smeru
  - Ovo rezultuje nepoželjnim oscilujućim efektom u ažuriranju gradijenata
- Zato centralizujemo x oko nule – kako bismo imali i pozitivne i negativne vrednosti

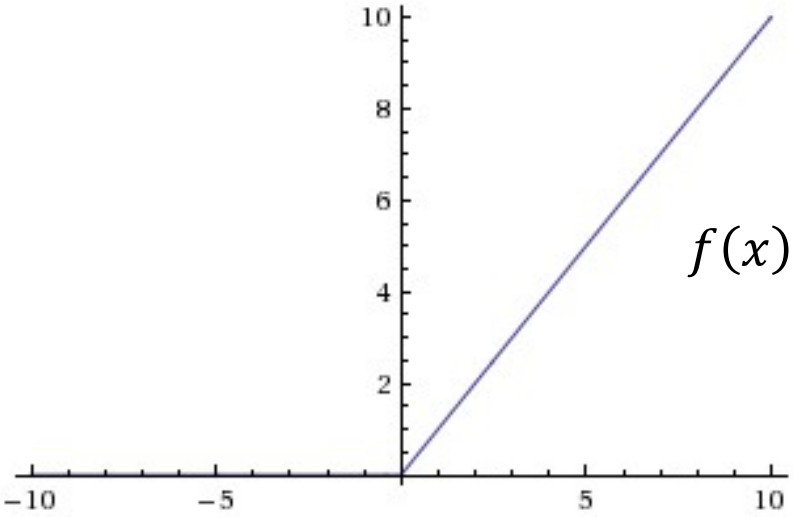


Tanh



- Mapira realan broj u opseg $[-1,1]$
- Kao i kod sigmoida, dolazi do saturacije aktivacija
- Za razliku od sigmoida je centriran oko nule
- Zato se u praksi uvek preferira naspram sigmoida

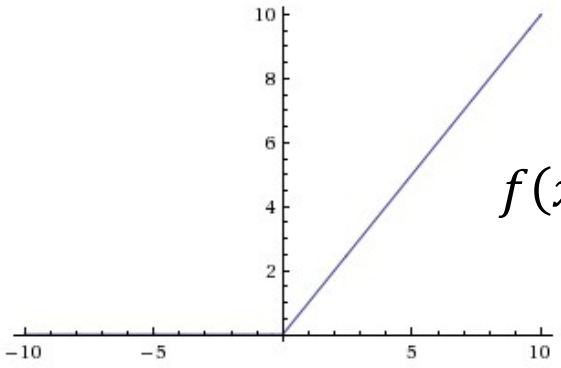
ReLU (*Rectified Linear Unit*)



$$f(x) = \max(0, x)$$

- Veoma popularan izbor
- Neće saturisati u pozitivnoj regiji
- U praksi mnogo brže konvergira od sigmoida i tahn
- Bolja aproksimacija onog što se dešava kod bioloških neurona
- Brz

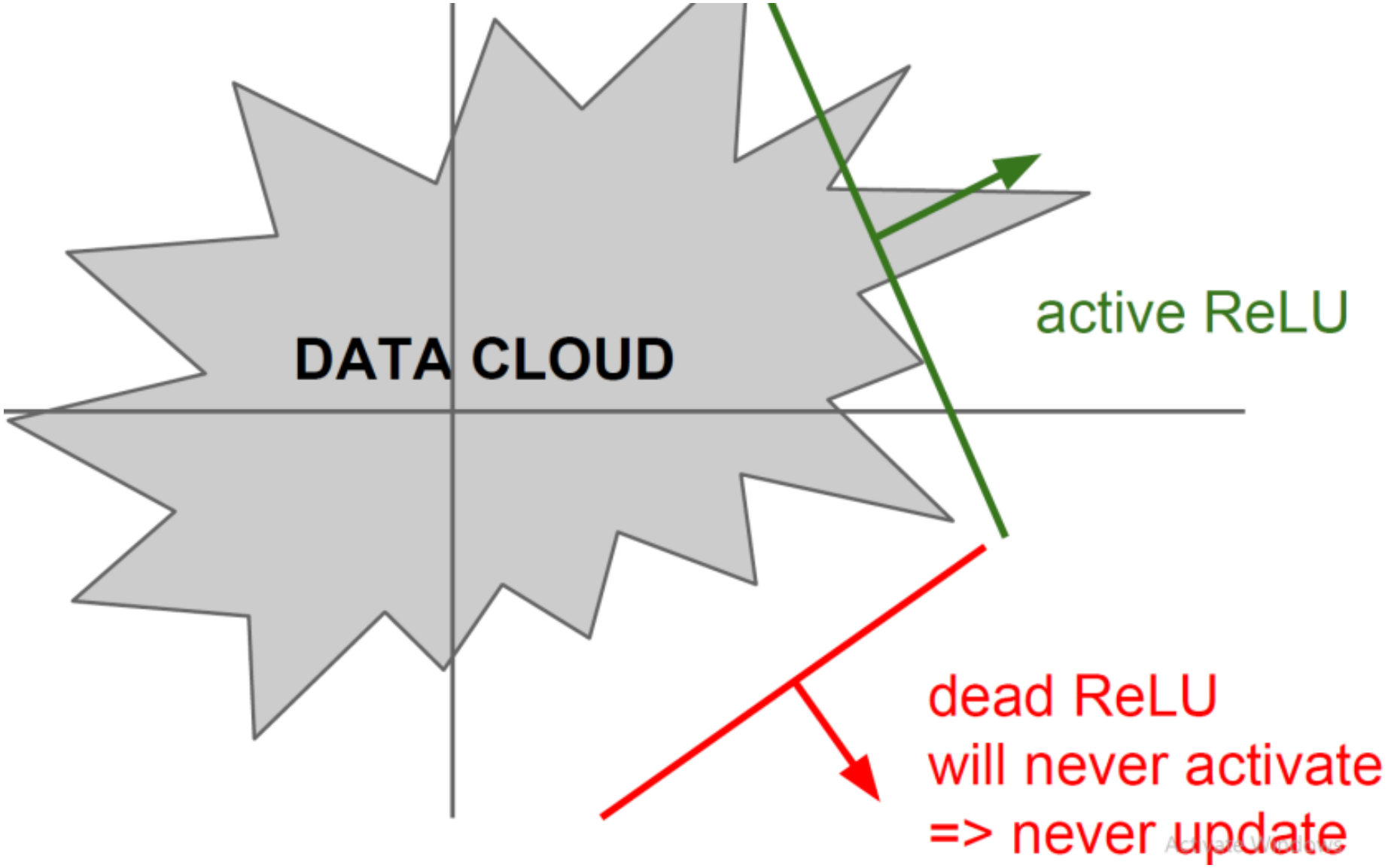
ReLU – problemi – „umirući“ neuroni



$$f(x) = \max(0, w^T x + b)$$

- Razmotrite slučaj kada je *bias b* velika negativna vrednost
- Može da ispadne da je $w^T x + b < 0$ za svako x. Gradijent ReLu aktivacije tokom propagacije u nazad je 0
- Odnosno, težine i *bias* koji su uzrokovali negativne predikcije se nikada ne ažuriraju
- Saturacija, slično kao kod sigmoida

ReLU – problemi – „umirući“ neuroni

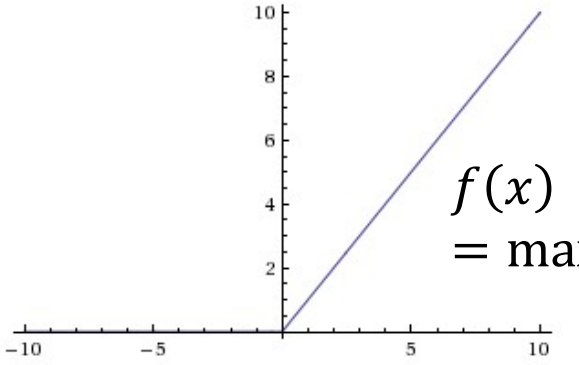


ReLU – problemi – „umirući“ neuroni

- Može biti posledica loše inicijalizacije – nikada ne dobijemo ulaz koji ih aktivira
- U praksi, neuroni se inicijalizuju sa blago pozitivnim bijasima (npr. 0.01)
- Može biti posledica velikog *learning rate-a*

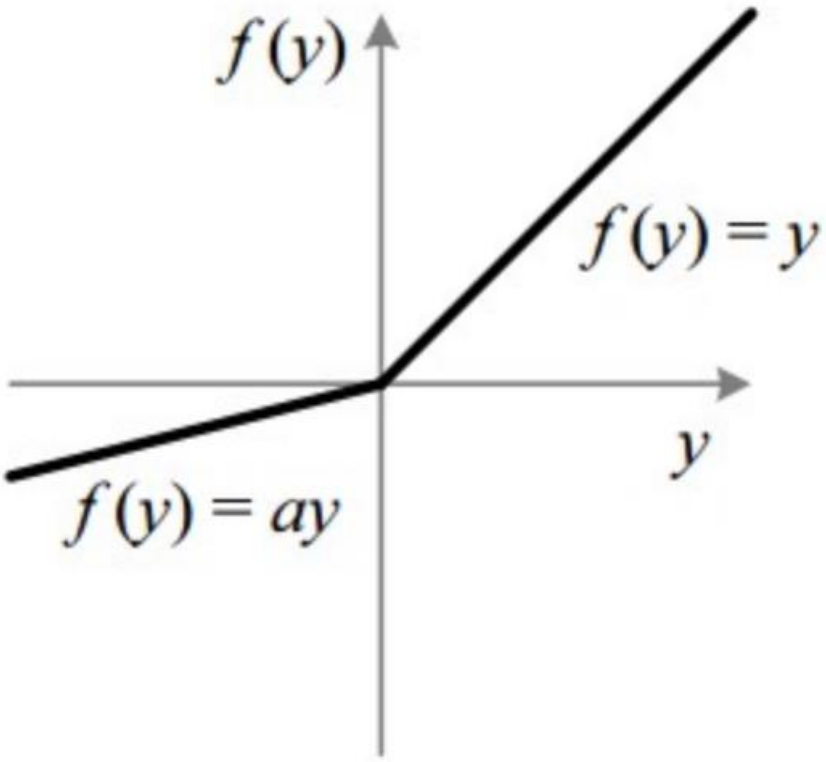
ReLU – problemi – nije centriran oko 0

- Nije centriran oko nule



- Izlaz je nenegativan, nezavisno od ulaza
- Ažuriranje svih ω će biti ili pozitivno ili negativno

Leaky ReLU



- Ne dolazi do saturacije
- Efikasno izračunavanje
- Konvergira mnogo brže od sigmoida i tanh
- Neuroni ne „umiru“

- Leaky ReLU
$$f(x) = \max(0.01x, x)$$

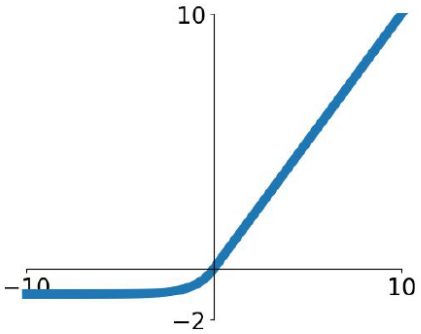
- Parametric Rectifier (PReLU)
$$f(x) = \max(\alpha x, x)$$

ELU

Activation Functions

[Clevert et al., 2015]

Exponential Linear Units (ELU)



- All benefits of ReLU
- Closer to zero mean outputs
- Negative saturation regime compared with Leaky ReLU adds some robustness to noise

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

- Computation requires exp()

Maxout

Maxout "Neuron" [Goodfellow et al., 2013]

- Does not have the basic form of dot product -> nonlinearity
- Generalizes ReLU and Leaky ReLU
- Linear Regime! Does not saturate! Does not die!

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

Problem: doubles the number of parameters/neuron :(

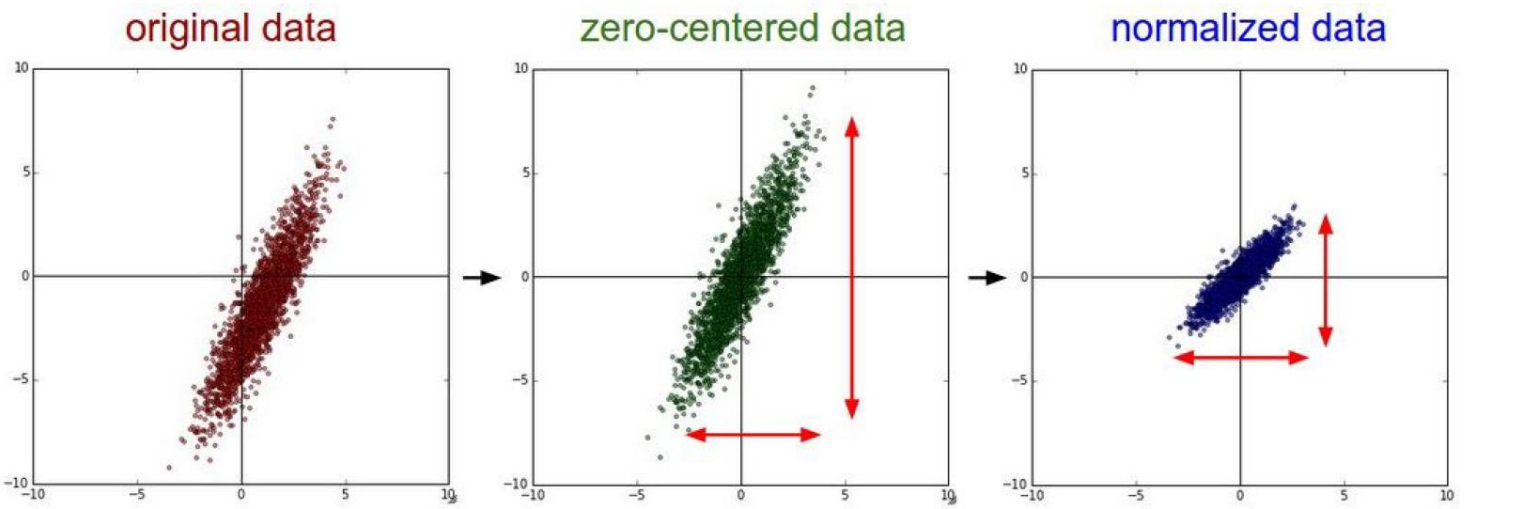
Kako odabrati aktivacionu funkciju

1. Probajte ReLU (pažljivo odaberite *learning rate*) – najbolje je krenuti od jednostavnih stvari
2. Ako ReLU ne daje obećavajuće rezultate, probajte Leaky ReLu ili ELU

- Nemojte koristiti *sigmoid* i *tahn*

Pretprocesiranje podataka

- Za slike, centriramo oko 0, ali nije neophodna normalizacija – intenziteti piksela se kreću u uporedivim opsezima



(Assume X [NxD] is data matrix,
each example in a row)

Inicijalizacija težina

- Neka je aktivaciona funkcija tanh

- Šta ako postavimo $w = 0$?
  - Svi neuroni će imati istu izlaznu vrednost, dobiće isti gradijent i ažuriraće se svi na isti način
  - Ne možemo razbiti simetriju

- Mali slučajni brojevi
  - Gausijani sa srednjom vrednošću 0 u standardnom devijacijom 0.01
  - Radi za manje mreže, ali ne najbolje za duboke. Pošto u svakom sloju množimo sa malim w, aktivacije će u dubljim slojevima postati bliske 0

- Veći slučajni brojevi (1 umesto 0.01)
  - Gotovo svi neuroni su saturisani, ili -1 ili +1 – svi gradijenti će biti 0

Inicijalizacija težina

- Xavier initialization [Glorot et al., 2010]
  - `W = np.random.randn(fan_in, fan_out)/np.sqrt(fan_in)`
  - Ali ne radi za ReLU

- He et al. 2015
  - `W = np.random.randn(fan_in, fan_out)/np.sqrt(fan_in/2)`

- U praksi, inicijalizacija je veoma važna
  - Može da učini razliku između toga da se mreža lepo trenira naspram toga da se ništa ne desi
  - Probajte Xavier, ako ne radi – isprobajte druge metode

Inicijalizacija težina

Proper initialization is an active area of research…

Understanding the difficulty of training deep feedforward neural networks by Glorot and Bengio, 2010

Exact solutions to the nonlinear dynamics of learning in deep linear neural networks by Saxe et al, 2013

Random walk initialization for training very deep feedforward networks by Sussillo and Abbott, 2014

Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification by He et al., 2015

Data-dependent Initializations of Convolutional Neural Networks by Krähenbühl et al., 2015

All you need is a good init, Mishkin and Matas, 2015

…